

Intelligent Agents & AI Categories

Recap

-Definition of AI

-Turing Test

-Expert Systems

-Machine Learning

-Deep Learning

-AI vs. Classical Programming

-Intelligent Agents

Components of an Intelligent Agent

- Sensors
- Actuators
- Perception
- Actions

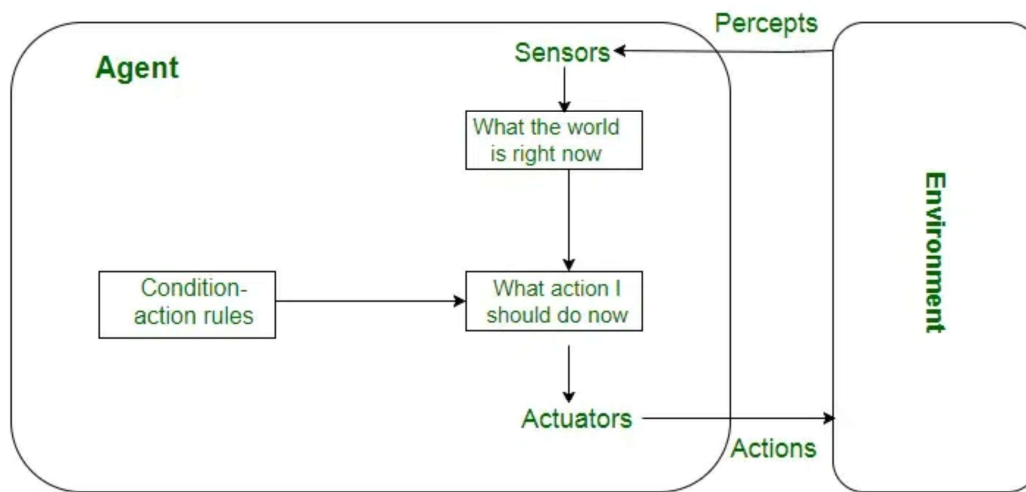
- **Sensors** are devices that allow the agent to perceive its environment by collecting data (e.g., cameras, microphones, temperature sensors).
- **Actuators** are mechanisms through which the agent acts on the environment (e.g., wheels, robotic arms, speakers).
- **Perception** refers to the interpretation of sensor data, enabling the agent to form an understanding of its environment.
- **Actions** are the decisions or operations the agent performs using its actuators to affect the environment.

Types of Agents

- Simple reflex
- Model-based
- Goal-based
- Utility-based
- Learning agents

- **Simple Reflex Agents:** Act only on the current percept. They do not retain any memory of past percepts. Example: A thermostat turning on/off based on current temperature.
- **Model-Based Reflex Agents:** Maintain an internal state to track aspects of the world not immediately perceptible. Example: A robot vacuum using a room map to avoid repeated cleaning.
- **Goal-Based Agents:** Decide actions based on a goal, enabling more flexibility than reflex agents. Example: A GPS system determining the best route to a destination.
- **Utility-Based Agents:** Consider multiple possible goals and choose actions that maximize a utility function. Example: A recommendation engine showing products that maximize click-through and satisfaction.
- **Learning Agents:** Improve their performance over time based on feedback. They have components like learning elements and performance elements. Example: A speech recognition system that improves its accuracy by learning from user corrections.

Simple Reflex Diagram



This agent selects actions based on the agent's current perception of the world and not based on past perceptions.

For example, if a Mars lander found a rock in a specific place it needed to collect, then it would collect it. If it was a simple reflex agent and found the same rock in a different place, it would still pick it up as it doesn't take into account that it already picked it up.

This is useful when a quick automated response is needed. Humans have a very similar reaction to fire; for example, our brain pulls our hand away without thinking about any possibility that there could be danger in the path of your arm. We call these reflex actions.

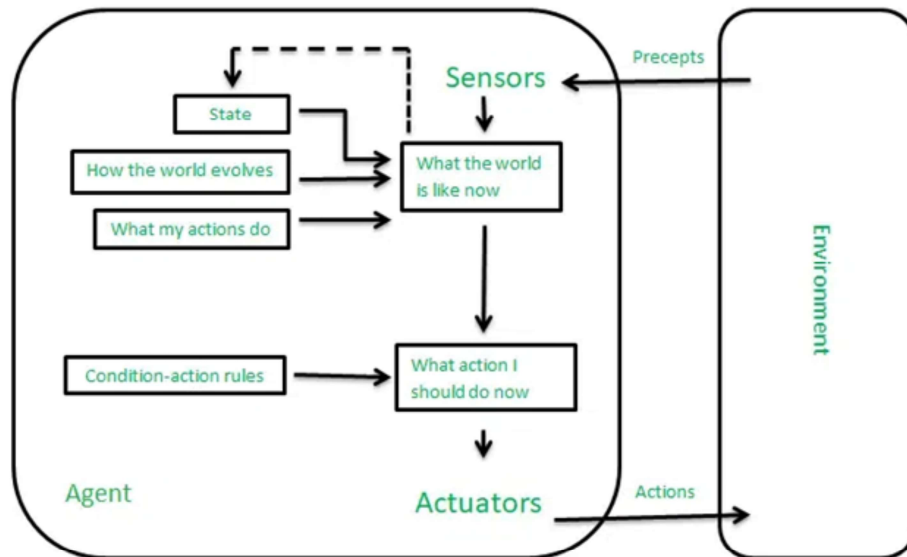
This kind of connection, where only one possibility is acted upon, is called a **condition-action rule**, written as:

if hand is in fire **then** pull away hand

The simple reflex agent has a library of such rules so that if a certain situation should arise and it is in the set of Condition-action rules, the agent will know how to react with minimal reasoning.

These agents are simple to work with but have very limited intelligence, such as picking up 2 rock samples.

Model-Based Diagram



Model-based reflex agents are made to deal with partial accessibility. They do this by keeping track of the part of the world it can see now. It does this by keeping an internal state that depends on what it has seen before so it holds information on the unobserved aspects of the current state.

This time our mars lander after picking up its first sample, it stores this in the internal state of the world around it so when it come across the second same sample it passes it by and saves space for other samples.

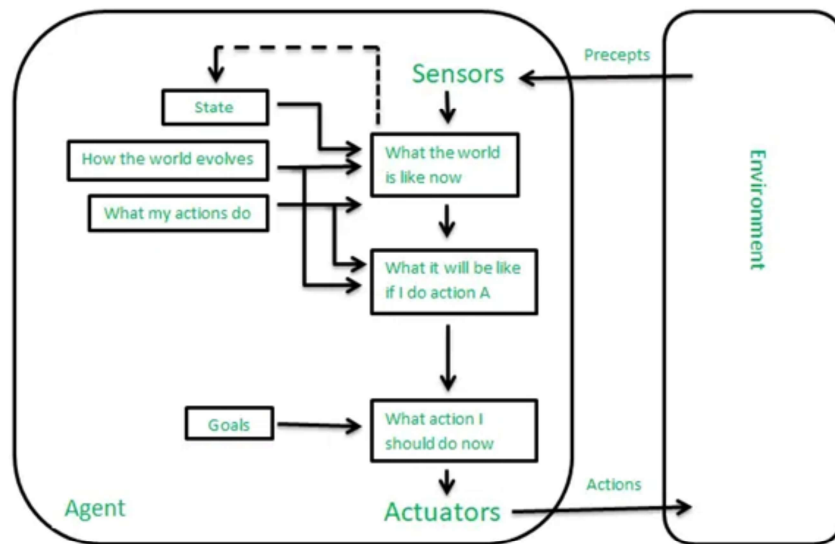
While reading this you are keeping track of where you have got to somewhere internally in your brain just in case you lose your place.

But in order to update this internal store we need 2 things:

1. Information on how the world evolves on its own.
e.g. If our mars Lander picked up the rock next to the one it was going to the world around it would carry on as normal
1. How the world is affected by the agents actions.
e.g. If our mars Lander took a sample under a precarious ledge it could displace a rock and it could be crushed.

We can predict how the world will react with facts like if you remove a supporting rock under a ledge the ledge will fall, such facts are called models, hence the name model-based agent.

Goal-Based Diagram



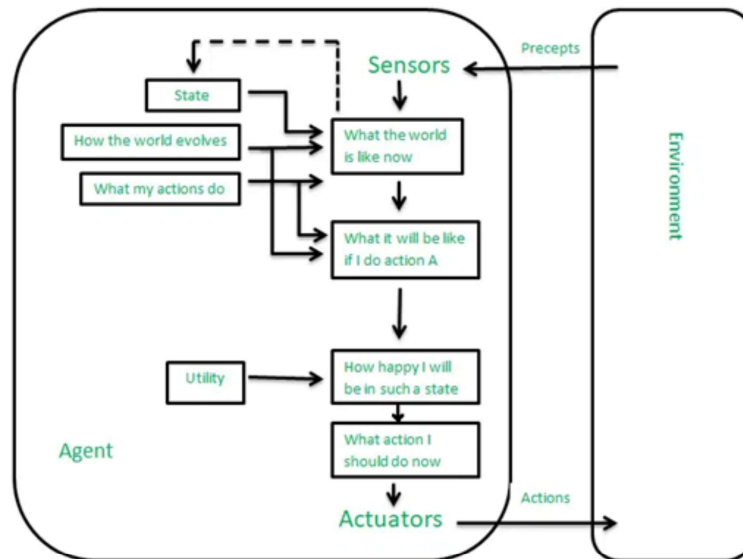
In life, in order to get things done we set goals for us to achieve, this pushes us to make the right decisions when we need to. A simple example would be the shopping list; our goal is to pick up everything on that list. This makes it easier to decide if you need to choose between milk and orange juice because you can only afford one. As milk is a goal on our shopping list and the orange juice is not, we chose the milk.

So in an intelligent agent, having a set of goals with desirable situations are needed. The agent can use these goals with a set of actions and their predicted outcomes to see which action/s achieve our goal/s.

Achieving goals can take 1 action or many actions. Search and planning are two subfields in AI devoted to finding sequences of actions to achieve an agent's goals.

Unlike the previous reflex agent, before acting, this agent reviews many actions and chooses the one which come closest to achieving its goals, whereas the reflex agents just have an automated response for certain situations. Although the goal-based agent does a lot more work than the reflex agent, this makes it much more flexible because the knowledge used for decision making is represented explicitly and can be modified. For example if our mars lander needed to get up a hill, the agent can update it's knowledge on how much power to put into the wheels to gain certain speeds. Through this, all relevant behaviors will now automatically follow the new knowledge on moving. However, in a reflex agent, many condition-action rules would have to be written.

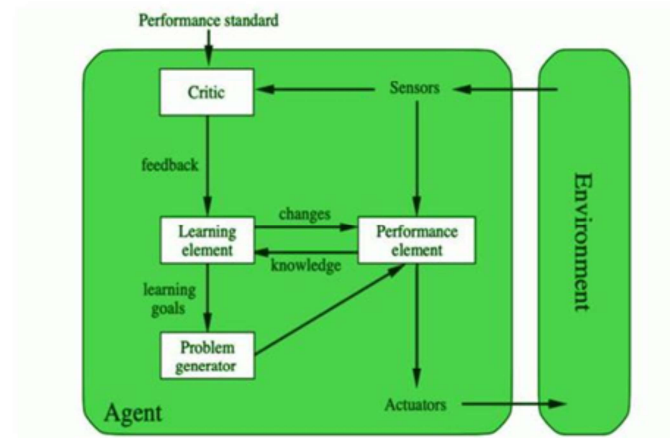
Utility-Based Diagram



Just having goals isn't good enough because often we may have several actions which all satisfy our goal so we need some way of working out the most efficient one. A utility function maps each state after each action to a real number representing how efficiently each action achieves the goal. This is useful when we either have many actions all solving the same goal or when we have many goals that can be satisfied and we need to choose an action to perform.

For example, let's show our mars lander on the surface of mars with an obstacle in its way. In a goal based agent, it is uncertain which path will be taken by the agent and some are clearly not as efficient as others. But in a utility based agent, the best path will have the best output from the utility function and that path will be chosen.

Learning Agent Diagram



When we expand our environments we get a larger and larger amount of tasks, eventually we are going to have a very large number of actions to pre-define. Another way of going about creating an agent is to get it to learn new actions as it goes about its business. This still requires some initial knowledge but cuts down on the programming greatly. This will allow the agent to work in environments that are unknown.

A learning agent can be split into the 4 parts shown in the diagram.

The learning element is responsible for improvements this can make a change to any of the knowledge components in the agents. One way of learning is to observe pairs of successive states in the percept sequence. From this, the agent can learn how the world evolves.

The performance element is responsible for selecting external actions, and this is considered to be the previous agents discussed.

For learning based agents, an external performance standard is needed to tell the critic if the agent's action has a good or a bad effect on the world. The learning agent gains feedback from the critic on how well the agent is doing and determines how the performance element should be modified, if at all, to improve the agent.

For example, in school, you would do a test and it would be marked. The test is the

critic. The teacher would mark the test and see what could be improved and instructs you how to do better next time. The teacher is the learning element and you are the performance element.

The last component is the problem generator. The performance element only suggests actions that it can already do, so we need a way of getting the agent to experience new situations, and this is what the problem generator is for. This way, the agent keeps on learning.

For example the school analogy. In science, with your current knowledge at that time you would not have thought of placing a mass on a spring. But, the teacher suggested an experiment and you did it and this taught you more and added to knowledge base.

PEAS Framework

- Performance
- Environment
- Actuators
- Sensors

This framework is used to specify the setting in which an intelligent agent operates. It helps in formally defining the task environment of an agent, which is critical to understanding and designing effective AI systems.

- **Performance:** The criteria for success of the agent's behavior. For example, in a vacuum cleaner robot, the amount of dirt cleaned or time taken.
- **Environment:** The external context in which the agent operates. For the vacuum cleaner, it could be a room with carpets and furniture.
- **Actuators:** The mechanisms through which the agent interacts with its environment (e.g., wheels, suction devices).
- **Sensors:** The components used to gather information from the environment (e.g., infrared sensors, dirt detectors).

This decomposition encourages a modular and structured approach to agent design, and is a foundational principle in AI systems engineering.

AI Categories

- Reactive Machines
- Limited Memory
- Theory of Mind
- Self-aware AI

- **Reactive Machines:** These are the most basic types of AI systems. They do not store memories or past experiences to inform future decisions. They simply react to the current input. Example: IBM's Deep Blue chess-playing computer.
- **Limited Memory:** These systems can use past experiences or historical data for a short duration. Most current AI applications (e.g., self-driving cars) fall under this category. They use sensors and historical data to make real-time decisions.
- **Theory of Mind:** This type of AI, still theoretical, would understand emotions, beliefs, intentions, and other mental processes of humans. It would be able to interact socially and empathetically with people.
- **Self-Aware AI:** This is the most advanced form of AI, which exists only in theory as of now. It would possess self-consciousness and an understanding of its own internal states, similar to human awareness

As of now, most real-world AI applications fall under the Reactive Machines and Limited Memory categories.

Activity 2

1. Design an intelligent agent for a smart home system.
2. Use PEAS to describe it.
3. Present your intelligent agent briefly (60 seconds max) in our online class.

1. This activity is individual.
2. Design an intelligent agent (see page 3 for guidance) for a smart home system by creating a diagram (see pages 5-9 for examples). Specify which part of your home you are going to use.
3. Use PEAS to describe it. See page 10 for guidance.
4. Present your intelligent agent briefly (60 seconds max) in our online class by sharing your screen and presenting your diagram. Describe your diagram using the PEAS Framework.
5. Similar systems are allowed, but please make yours unique to you.
6. Submit a 1-page (max) PDF document detailing your intelligent agent.
 - a. Include the class code, name of the class, your name, your section, and date of document creation.
 - b. Font style: Times New Roman; Font size: 12; single spacing, remove space before and after paragraph.
7. Deadline of submission is on July 8, 2025, until the end of our online class.

Mini Project 1: Chatbot

1. Group yourselves into 3.
2. Create a **simple chatbot** (huvað mag overthink) that responds to greetings and questions about the weather or time.
3. Starter Python code is provided below. Please make your chatbot **unique** to your group.
4. Deadline: July 21, 2025, during our F2F class.

```
import datetime

def get_response(user_input):
    user_input = user_input.lower()
    if "hello" in user_input or "hi" in user_input:
        return "Hello! How can I assist you today?"
    elif "time" in user_input:
        current_time = datetime.datetime.now().strftime("%H:%M")
        return f"The current time is {current_time}."
    elif "weather" in user_input:
        return "I'm not connected to the internet, but let's pretend it's sunny today!"
    else:
        return "Sorry, I can only help with greetings, time, or weather questions."

# Main chat loop
print("Chatbot: Hello! Type 'bye' to exit.")
while True:
    user_input = input("You: ")
    if user_input.lower() == "bye":
        print("Chatbot: Goodbye!")
        break
    response = get_response(user_input)
    print("Chatbot:", response)
```